

Case Study: MATLAB analysis on Multi Spring Mass Static Equilibrium

Objective: Form stiffness matrix for 5 to 10 mass spring chain and compute displacement under applied loads using linear system solvers. Plot the displacement v/s time graph for each mass.

Comprehensive Description of Multi Spring–Mass System with Damping and Friction

1. System Description

The code models a 1-D multi-degree-of-freedom spring–mass system.

- There are N masses placed in a straight line.
- Each adjacent mass is connected by a linear spring.
- The first mass is connected to the left wall using a spring of stiffness k_0 .
- The last mass is connected to the right wall using a spring of stiffness k_N .
- A force is applied to the first mass.
- The masses move horizontally on a surface.
- Friction exists between the masses and the floor.
- Viscous damping is present in the system.

This creates a realistic mechanical model similar to connected vibration absorbers, suspension chains, or distributed mechanical systems.

2. Assumptions Taken

To simplify and model the system properly, the following assumptions are made:

1. Motion is **only in one direction (horizontal)**.
2. Springs are **linear** and follow Hooke's Law.
3. Damping is **viscous**, proportional to velocity.
4. Friction is **Coulomb friction**, proportional to μmg .
5. Masses are treated as **point masses**.
6. The surface is flat and gravity is constant.
7. No rotational motion exists.
8. System response is solved numerically using time stepping.

These assumptions make the system mathematically manageable while still capturing real engineering behaviour.

3. Mathematical Model of the System

The dynamic equation governing the system is:

$$M\ddot{x} + C\dot{x} + Kx + F_{\text{fric}} = F$$

Where:

- (M) = Mass matrix (diagonal)
- (C) = Damping matrix

- (K) = Stiffness matrix
- (x) = Displacement vector
- $\dot{x}$ = Velocity vector
- $\ddot{x}$ = Acceleration vector
- F_{fric} = Friction force vector
- F = External force vector

Mass Matrix (M)

$$M = \text{diag}(m_1, m_2, \dots, m_N)$$

Each diagonal element represents the inertia of a mass. Since masses do not directly affect each other through inertia, there are no off-diagonal terms.

Damping Matrix (C)

$$C = c * I$$

Assumes equal viscous damping for each mass.

Stiffness Matrix (K)

The stiffness matrix is assembled based on spring connections.

It follows a **tridiagonal symmetric pattern**:

- For internal masses:

$$K(i, i) = k_{i-1} + k_i$$

- Off-diagonal terms:

$$K(i, i - 1) = -k_{i-1}$$

$$K(i, i + 1) = -k_i$$

For boundary masses:

First mass:

$$K(1, 1) = k_0 + k_1$$

Last mass:

$$K(N, N) = k_{N-1} + k_N$$

Pattern of Stiffness Matrix

It has this structure:

$$\begin{bmatrix} k_0 + k_1 & -k_1 & 0 & \dots \\ -k_1 & k_1 + k_2 & -k_2 & \dots \\ 0 & -k_2 & k_2 + k_3 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Key Characteristics:

- Symmetric
- Sparse
- Tridiagonal

- Diagonally dominant
- Physically represents force balance at each mass

This structure is typical in vibration and finite element problems.

4. Friction Model:

Friction is modelled as:

$$F_{fric} = \mu mg \cdot \text{sign}(v)$$

- Opposes direction of motion
- Depends on velocity sign
- Acts independently on each mass
- Makes system nonlinear

5. Time Integration Method:

The system is solved using **explicit time stepping** (Euler method).

Acceleration is calculated using:

$$a = M^{-1}(F - Cv - Kx - F_{fric})$$

Then:

$$v_{new} = v + dt \cdot a$$

$$x_{new} = x + dt \cdot v$$

This updates displacement and velocity step by step.

6. How Loops and Conditionals Work

For loops are used to:

1. Collect mass values from the user.
2. Collect stiffness values of springs.
3. Construct the stiffness matrix row by row depending on position of each mass.
4. Run the simulation across many time steps.

The time loop continues until steady state is reached or the maximum number of steps is completed.

If conditions are used in two main areas:

1. During stiffness matrix construction to treat the first and last masses differently from internal masses.
2. During simulation to check whether the system has reached steady state.

The steady state condition verifies whether:

- The maximum velocity is below a defined tolerance.

- The maximum acceleration is below a defined tolerance.

If both values are sufficiently small, the system is considered stabilized and the loop stops.

7. Time Stamp and Displacement vs Time

Time Tracking

$$t(\text{step}+1) = t(\text{step}) + dt;$$

This increases time by a fixed increment dt at each iteration.

At the same time:

- Displacement values are stored for each mass.
- Velocity values are stored.
- Acceleration values are stored.

After simulation completes, the stored time array and displacement matrix are used to generate the displacement versus time graph.

This graph clearly shows how each mass moves from its initial condition until equilibrium is reached.

Correction in Friction Modelling

The earlier implementation used a discontinuous Coulomb friction model based on the sign of velocity:

$$F_{fric} = \mu mg \cdot \text{sign}(v)$$

This caused numerical instability at very low velocities due to the abrupt change in friction force near $v = 0$, leading to non-physical initial displacement behaviour.

To resolve this, the friction model was modified to a smooth approximation:

$$F_{fric} = \mu mg \cdot \tanh(\alpha v)$$

where α is a large constant.

This modification:

- Eliminates discontinuity at zero velocity
- Ensures smooth transition between static and kinetic behaviour
- Improves numerical stability

The displacement update equation was modified from the standard explicit Euler method to a semi-implicit approach:

Previous form:

$$x_{n+1} = x_n + dt \cdot v_n$$

Updated form:

$$x_{n+1} = x_n + dt \cdot v_{n+1}$$

This change enhances:

- Stability of the numerical solution
- Accuracy in dynamic response
- Suitability for stiff spring-mass systems

8. Engineering Interpretation of Results

The graph shows:

1. Initial acceleration due to applied force.
2. Oscillatory behaviour due to springs.
3. Energy loss due to damping.
4. Motion resistance due to friction.
5. Gradual stabilization to static equilibrium.

Final displacement:

$$Kx_{\text{final}} = F - F_{\text{fric}}$$

Thus, the system settles into a balance of:

- Spring forces
- Friction forces
- Applied force

9. Engineering Importance:

This model represents:

- Multi-body vibration systems
- Structural dynamic chains
- Vehicle suspension modelling
- Conveyor or linked mechanical assemblies
- Finite element discretized structures

It combines:

- Mass inertia
- Elastic restoring force
- Energy dissipation (damping)
- Nonlinear friction
- External loading

The program provides a complete numerical simulation of real mechanical behaviour in a controlled and scalable way.

10. Conclusion:

This MATLAB code successfully models a realistic multi-degree-of-freedom spring-mass system with:

- Boundary springs
- Inter-mass springs
- Viscous damping
- Coulomb friction
- External force
- Time-domain numerical solution

- The stiffness matrix follows a structured tridiagonal form typical of connected mechanical systems.
- The time-stepping loop simulates real-time motion evolution.
- The stopping condition ensures detection of steady state.

Overall, the code is a complete engineering simulation tool for studying dynamic behaviour of interconnected mechanical systems under damping and friction effects.

Code:

```
1 % Multi Spring Mass Static Equilibrium
2 % Form stiffness matrix for 5 to 10 mass spring chain and compute displacement under
3 % applied loads using linear system solvers.
4
5 clc;
6 clear;
7 close all;
8
9 N = input('Enter number of masses: ');
10
11 m = zeros(N,1);
12 disp('Enter mass values (kg):');
13
14 % mass input from user
15 for i = 1:N
16     m(i) = input(['m(', num2str(i), ') = ']);
17 end
18
19 % constants input from user
20 c = input('Enter damping coefficient (Ns/m): ');
21 mu = input('Enter coefficient of friction (0 to 1): ');
22 g = 9.81;
23 k0 = input('Enter stiffness of LEFT wall spring (N/m): ');
24
25 % stiffness input from user
26 k = zeros(N-1,1);
27 for i = 1:N-1
28     k(i) = input(['k(', num2str(i), ') = ']);
29 end
30
31 kN = input('Enter stiffness of RIGHT wall spring (N/m): ');
32
33 F1 = input('Enter load applied on first mass (N): ');
34
35 M = diag(m);
36 C = c * eye(N);
37 K = zeros(N,N);
38
39 % Stiffness matrix of spring system
40 for i = 1:N
41     if i == 1
42         K(i,i) = k0 + k(1);
43         K(i,i+1) = -k(1);
44     elseif i == N
45         K(i,i) = k(N-1) + kN;
46         K(i,i-1) = -k(N-1);
47     else
```

```

48         K(i,i) = k(i-1) + k(i);
49         K(i,i-1) = -k(i-1);
50         K(i,i+1) = -k(i);
51     end
52 end
53
54 % force matrix
55 F = zeros(N,1);
56 F(1) = F1;
57
58 % time stamp
59 dt = 0.001;
60 max_steps = 2000;
61 tolerance = 1e-6;
62
63 t = zeros(1,max_steps);
64 x = zeros(N,max_steps);
65 v = zeros(N,max_steps);
66 a = zeros(N,max_steps);
67
68 for step = 1:max_steps-1
69
70     t(step+1) = t(step) + dt;
71
72     F_fric = mu * m * g .* tanh(100 * v(:,step));
73
74     a(:,step) = M \ (F - C*v(:,step) - K*x(:,step) - F_fric);
75
76     v(:,step+1) = v(:,step) + dt*a(:,step);
77     x(:,step+1) = x(:,step) + dt*v(:,step+1);
78
79     if max(abs(v(:,step+1))) < tolerance && ...
80        max(abs(a(:,step))) < tolerance
81
82         disp(['System reached steady state at time = ', num2str(t(step+1)), ' seconds']);
83         break
84     end
85 end
86
87 x = x(:,1:step+1);
88 t = t(1:step+1);
89
90 figure;
91 plot(t, x, 'LineWidth', 1.5);
92 xlabel('Time (s)');
93 ylabel('Displacement (m)');
94 title('Displacement of Masses vs Time');

```

```

95     grid on;
96
97     legendStrings = strings(1,N);
98     for i = 1:N
99         legendStrings(i) = ['Mass ', num2str(i)];
100     end
101     legend(legendStrings);
102
103     x_final = x(:,end);
104
105     disp('Final displacement of masses (m):');
106     disp(x_final);
107
108     spring_ext = zeros(N+1,1);
109     spring_ext(1) = x_final(1);
110
111     for i = 1:N-1
112         spring_ext(i+1) = x_final(i+1) - x_final(i);
113     end
114
115     spring_ext(N+1) = -x_final(N);
116
117     disp('Final displacement (extension) of springs (m):');
118     disp(spring_ext);

```

Figure 1 MATLAB Code

Result:

Command Window

```
Enter number of masses: 5
Enter mass values (kg):
m(1) = 50
m(2) = 100
m(3) = 150
m(4) = 200
m(5) = 250
Enter damping coefficient (Ns/m): 3462
Enter coefficient of friction (0 to 1): 0.4
Enter stiffness of LEFT wall spring (N/m): 546
k(1) = 534
k(2) = 567
k(3) = 768
k(4) = 754
Enter stiffness of RIGHT wall spring (N/m): 253
Enter load applied on first mass (N): 5462
Final displacement of masses (m):
    6.1251
    2.8075
    0.7262
    0.1230
    0.0401

Final displacement (extension) of springs (m):
    6.1251
   -3.3175
   -2.0814
   -0.6031
   -0.0829
   -0.0401
```

Figure 2 MATLAB Result

Graph:

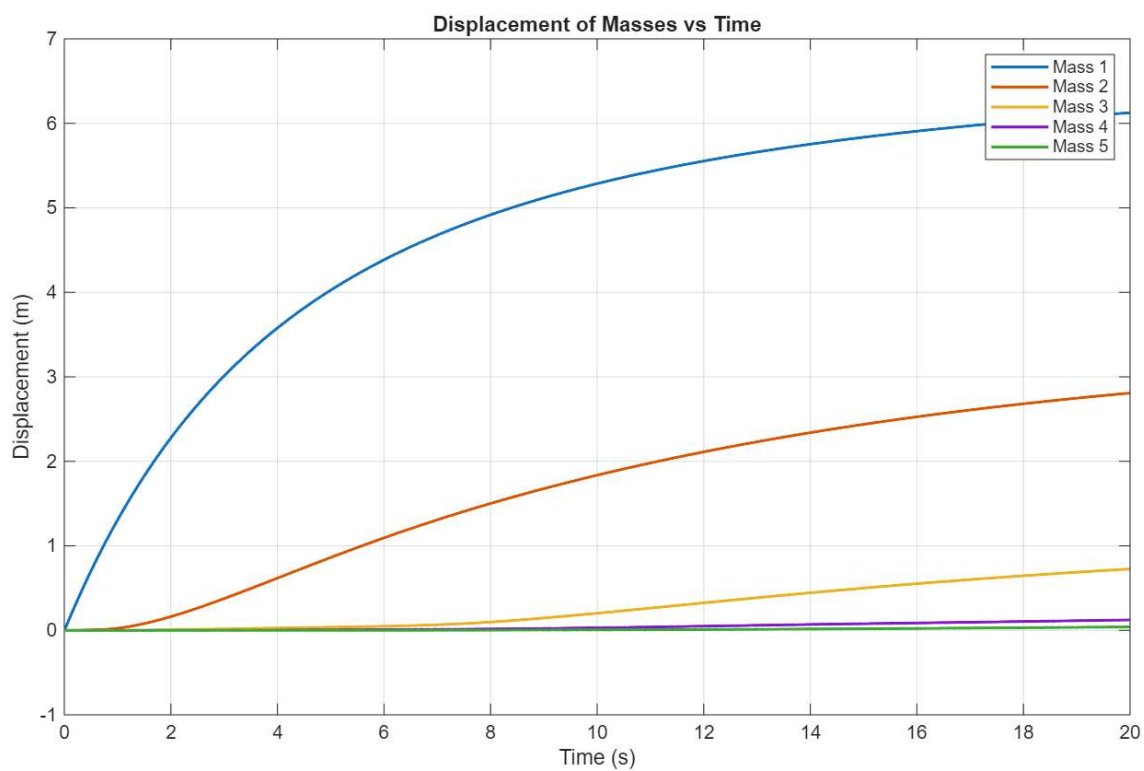


Figure 3 Displacement (m) V/S Time (Sec) graph